

HOPPER INFORMATION SERIES

06 / COMPUTER VISION

Object detection and COCO

How the built-in local neural network names and locates common objects.



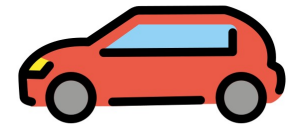
airplane



apple



banana



car

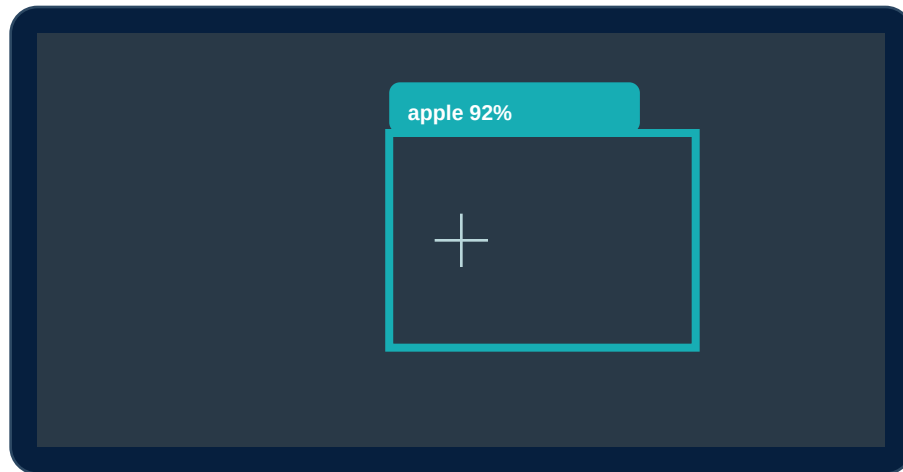




Detection is not the same as classification

OBJECT DETECTOR

Find multiple known objects and report where each one is.



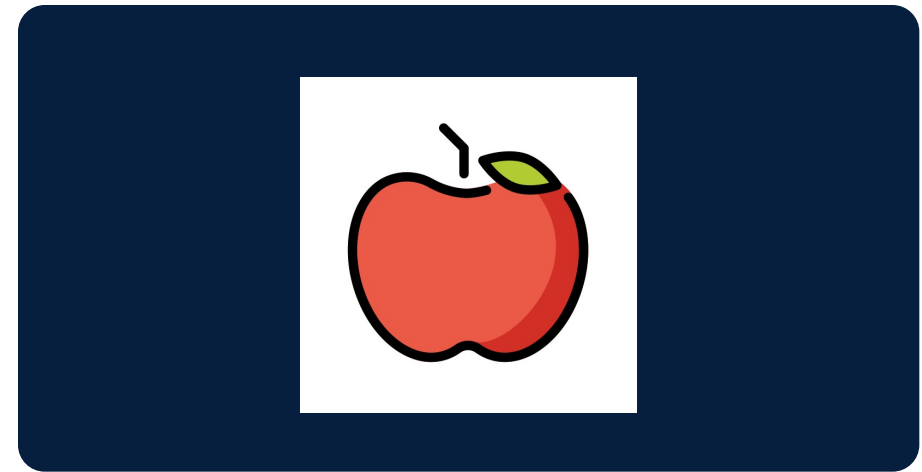
OUTPUT

class + confidence + box + x/y

Built-in COCO-SSD

IMAGE CLASSIFIER

Choose the best whole-frame class. It does not draw a box.



apple landing pad 91%

Custom Teachable Machine



COCO means Common Objects in Context

330K

IMAGES

>200K

LABELED IMAGES

1.5M

OBJECT INSTANCES

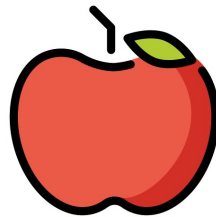
80

OBJECT CATEGORIES

Examples used in the Hopper simulator and recognized label set



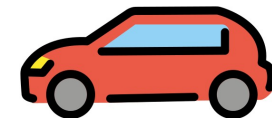
airplane



apple



banana

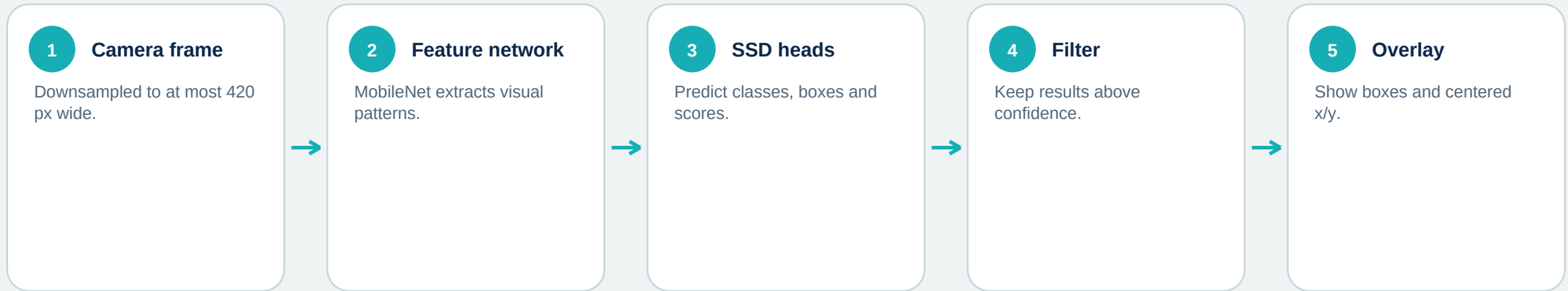


car

Illustrations are simulator targets, not COCO training photographs. OpenMoji CC BY-SA 4.0.



The built-in network runs locally



LAZY + LOCAL

`public/models/coco-ssd/model.json`

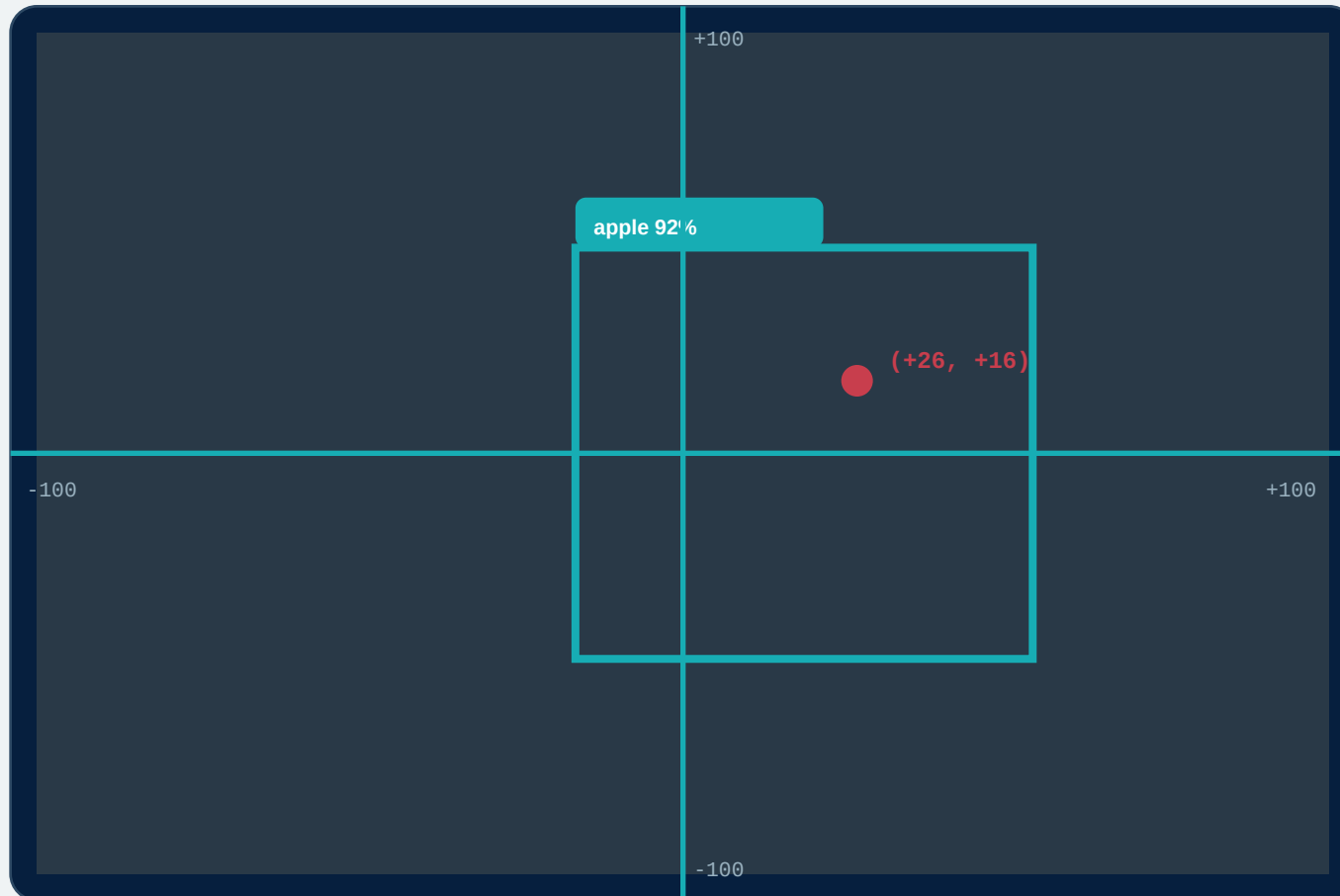
Loads only when object detection is requested. No image upload and no internet model fetch.

MAX 10 BOXES

DEFAULT 0.55



Read a detection like a measurement



DETECTION OBJECT

- class: exact COCO label
- score: 0..1
- bbox: left, top, width, height in pixels
- centerX / centerY: -100..100

COORDINATE RULE

- Frame center is (0,0).
- Right and up are positive.
- objectCoordinate reads last known state.
- 0 can mean absent or perfectly centered.



Scan, test, then use saved coordinates

scan for objects

camera sees apple at 55% confidence

x coordinate of apple at 55% confidence

OBJECT CHECK

```
1  const seen = await
2    vision.seesObject("apple", 0.55);
3  if (seen) {
4    const x = vision.objectCoordinate(
5      "apple", "x", 0.55);
6    console.log({ seen, x });
7  }
```

FRESH VS SAVED

- seesObject performs a fresh inference every call.
- objectCoordinate never scans; later misses do not clear its per-label last-known map.
- Use a fresh Boolean scan to establish current visibility before trusting x/y.



Design experiments around model limitations

MAKE COCO EASIER

- Use exact labels from the 80-class list.
- Choose large, well-lit, separated objects.
- Test distance, rotation and background.
- Tune confidence with false positives and misses.
- Require repeated consistent detections.

KNOW WHEN TO SWITCH

- pencil is not a COCO label.
- Small or thin objects are difficult.
- Occlusion and unusual viewpoints reduce confidence.
- Use Teachable Machine for classroom-specific whole-frame classes.

SOURCES AND FURTHER READING

COCO dataset

<https://cocodataset.org/>

Microsoft COCO paper

<https://arxiv.org/abs/1405.0312>

TensorFlow.js COCO-SSD

<https://github.com/tensorflow/tfjs-models/tree/master/coco-ssd>

Hopper Studio local model implementation

[lib/vision.ts](#) and [README.md](#)

OpenMoji simulator illustrations

<https://openmoji.org/> - CC BY-SA 4.0